

Java Backend Architecture

Interview Series

Chapter 7.9

Maven vs Gradle: Deep Comparison

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 7.9 — Maven vs Gradle: Deep Comparison

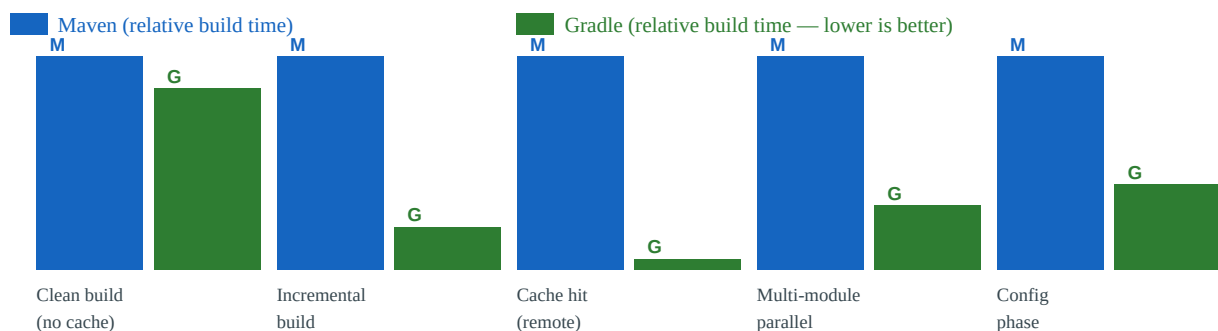
Introduction

Maven and Gradle are the two dominant build tools in the Java ecosystem. Maven offers stability, convention over configuration, and a vast plugin ecosystem built over 20+ years. Gradle offers superior performance, flexibility, and modern DSL support. Choosing between them — or migrating from one to the other — requires understanding their fundamental differences in philosophy, performance, and ecosystem maturity.

Feature Matrix

Feature	Maven	Gradle
Build language	XML (verbose)	Groovy/Kotlin DSL
Build speed	Slower (no caching)	Faster (cache+incremental)
Learning curve	Gentle (declarative)	Steeper (imperative)
IDE support	Excellent	Good (Kotlin DSL better)
Convention	Strong (opinionated)	Flexible (any structure)
Flexibility	Limited	Highly extensible
Ecosystem	Very mature	Growing fast
Android support	Not supported	Official build tool
Incremental compile	Limited	Full support
Build cache	No native cache	Local + remote cache

Performance Comparison



Gradle significantly outperforms Maven in most build scenarios through: (1) **incremental compilation** — only recompiles changed classes, (2) **up-to-date checks** — skips unchanged tasks, (3) **build cache** — reuses outputs from previous builds or other machines, (4) **parallel execution** — builds independent modules concurrently. For clean builds (no cache), the difference is smaller; with caching, Gradle can be 10-100x faster.

Build Script Comparison

```
<!-- Maven pom.xml --> <project> <modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId> <artifactId>my-app</artifactId> <version>1.0.0</version>
```

```
<properties> <maven.compiler.release>17</maven.compiler.release> </properties> <dependencies>
<dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId> <version>3.2.0</version> </dependency>
<dependency> <groupId>org.junit.jupiter</groupId> <artifactId>junit-jupiter</artifactId>
<version>5.10.1</version> <scope>test</scope> </dependency> </dependencies> </project>
```

```
// Gradle build.gradle.kts (equivalent) plugins { id("org.springframework.boot") version
"3.2.0" java } java { toolchain { languageVersion = JavaLanguageVersion.of(17) } } dependencies
{ implementation("org.springframework.boot:spring-boot-starter-web")
testImplementation("org.junit.jupiter:junit-jupiter:5.10.1") }
```

Dependency Resolution Strategy Comparison

Aspect	Maven	Gradle
Conflict resolution	Nearest-wins (closest to root)	Highest-version wins (default)
BOM import	scope=import in dependencyMgmt	platform() / enforcedPlatform()
Version constraints	No rich constraints	require / prefer / strictly / reject
Dependency locking	No native support	Native lockfiles (--write-locks)
Exclusions	<exclusion> per dependency	exclude() per dep or global
Optional deps	<optional>true</optional>	compileOnly or custom capability
Transitive scopes	Scope-based (compile/provided/runtime)	Configuration-based (api/impl)

Lifecycle vs Task Graph

Aspect	Maven Lifecycle	Gradle Task Graph
Model	Fixed phases in fixed order	Flexible DAG of tasks
Extensibility	Bind goals to existing phases only	Add/remove/order any task
Phase skipping	Cannot skip phases (only goals)	Any task can be skipped/replaced
Custom phases	Not supported	Create tasks at any point in graph
Execution model	Sequential phases per module	Parallel tasks across modules

Migration: Maven to Gradle

```
# Automatic migration tool (run in project root) gradle init --type pom # This: # 1. Reads
pom.xml and all module POMs # 2. Generates settings.gradle.kts # 3. Generates build.gradle.kts
for each module # 4. Migrates dependencies with correct configurations # 5. Creates Gradle
wrapper # After migration – verify: ./gradlew build ./gradlew dependencies ./gradlew --scan #
check build scan for issues # Common migration issues: # - Maven scope "compile" → Gradle
"implementation" (not api) # - Maven scope "provided" → Gradle "compileOnly" # - Maven scope
"runtime" → Gradle "runtimeOnly" # - Maven profiles → Gradle project properties or
conditional blocks # - Maven resource filtering → Gradle processResources { expand() } # -
Maven assembly → Gradle distribution or shadow plugin
```

Scope Mapping: Maven to Gradle

Maven Scope	Gradle Configuration	Notes
compile	implementation	Default mapping (prefer api only for public API types)

provided	compileOnly	Not in runtime classpath
runtime	runtimeOnly	Not in compile classpath
test	testImplementation	Test compile + runtime
system	(files())	Use local file dependency — system scope is deprecated
import	platform()	BOM import — use platform() or enforcedPlatform()

When to Choose Maven

Situation	Reason to Choose Maven
Enterprise Java EE / Jakarta EE	Maven is the established standard; extensive plugin support
Team unfamiliar with Groovy/Kotlin	XML is universally understood; lower learning curve
Strict conventional project structure	Maven enforces conventions more strictly
Existing Maven-based organisation	Consistency and shared infrastructure (Nexus, CI pipelines)
Audit/compliance requirements	Declarative XML is easier to audit and reason about

When to Choose Gradle

Situation	Reason to Choose Gradle
Android development	Gradle is the official Android build tool
Performance-critical CI	Build cache + incremental builds dramatically reduce CI time
Complex custom build logic	Full programming language (Kotlin/Groovy) for build scripts
Multi-repo / composite builds	includeBuild() for local substitution without publishing
New Spring Boot projects	Spring Initializr defaults to Gradle for new projects
Microservices with many modules	Parallel builds and build cache give huge time savings

50+ Interview Questions & Answers

Q1. What is the fundamental philosophical difference between Maven and Gradle?

Maven: convention over configuration — declarative XML, fixed lifecycle, strong opinions. Gradle: configuration over convention — programmatic DSL, flexible task graph, minimal built-in assumptions.

Q2. Which is faster, Maven or Gradle?

Gradle is significantly faster for most builds due to incremental compilation, up-to-date task skipping, build cache (local and remote), and parallel execution. For clean builds without cache, the difference is smaller.

Q3. What is Gradle's build cache and does Maven have an equivalent?

Gradle's build cache stores task outputs by input hash, enabling cache hits across machines and after clean builds. Maven has no native equivalent — each clean build re-executes everything.

Q4. What is the difference in dependency conflict resolution between Maven and Gradle?

Maven uses nearest-wins (fewest hops from root). Gradle uses highest-version-wins by default. Both allow explicit overrides. Gradle also supports rich version constraints (require/prefer/strictly/reject).

Q5. How does Maven's scope system compare to Gradle's configurations?

Maven scopes (compile/provided/runtime/test/import/system) map roughly to Gradle's configurations (implementation/compileOnly/runtimeOnly/testImplementation/platform/files). Gradle's system is more flexible with api vs implementation distinction.

Q6. What does gradle init --type pom do?

Reads existing pom.xml files and automatically generates equivalent Gradle build scripts (settings.gradle.kts, build.gradle.kts for each module, Gradle wrapper). Starting point for Maven-to-Gradle migration.

Q7. How do Maven profiles compare to Gradle?

Maven profiles are XML sections activated by flags or conditions. Gradle uses if/when blocks in build scripts, project properties (passed via -P), or convention plugins with conditional logic.

Q8. What is the equivalent of Maven's dependencyManagement in Gradle?

platform() imports a BOM for version management. Or define versions in libs.versions.toml version catalog. Or use resolutionStrategy.force(). There's no exact equivalent to parent POM dependencyManagement.

Q9. How do multi-module builds compare between Maven and Gradle?

Maven: in aggregator POM, -pl/-am/-amd flags, sequential by default. Gradle: include() in settings.gradle, :module:task syntax, parallel by default with org.gradle.parallel=true.

Q10. What is the equivalent of Maven's reactor in Gradle?

Gradle's task execution engine which builds a DAG of tasks considering project dependencies and runs them in correct order (with optional parallelism). More flexible than Maven's fixed lifecycle.

Q11. How does resource filtering compare between Maven and Gradle?

Maven: true in maven-resources-plugin — replaces \${property} in files. Gradle: processResources { expand(project.properties) } or filesMatching("*.properties") { expand(...) }.

Q12. How do you replicate Maven's assembly plugin in Gradle?

Use the distribution plugin (part of Gradle) for runnable distributions, or the Shadow plugin for uber-JARs. For complex assembly needs, define custom Copy/Zip tasks or use the Gradle Distribution plugin.

Q13. What replaces Maven's release plugin in Gradle?

The axion-release plugin or researchgate/release plugin provide similar tag-and-version-bump functionality. Many teams use CI/CD pipeline scripts instead of a release plugin.

Q14. How do you replicate Maven properties interpolation in Gradle?

In build.gradle.kts: val myVersion: String by project (from gradle.properties). In resources: processResources { expand(mapOf('version' to version)) }.

Q15. What is the equivalent of Maven's maven-compiler-plugin in Gradle?

The java plugin provides compileJava and compileTestJava tasks with sourceCompatibility/targetCompatibility. The toolchain {} block is the modern equivalent of parameter.

Q16. How do you run integration tests in Maven vs Gradle?

Maven: maven-failsafe-plugin bound to integration-test/verify phases. Gradle: create a separate sourceSet (integrationTest), add a Test task type, configure task dependencies. No built-in equivalent to Failsafe.

Q17. Which build tool has better IDE support?

Maven has historically had slightly better IDE support. Gradle with Kotlin DSL (build.gradle.kts) now has excellent IntelliJ IDEA support with full autocomplete, navigation, and refactoring. Groovy DSL has limited

autocomplete.

Q18. How does test parallelism compare?

Both support parallel test execution. Maven Surefire: `methods4`. Gradle: `tasks.withType { maxParallelForks = 4 }`. Gradle also supports parallel task execution across subprojects.

Q19. What is the Gradle equivalent of Maven's BOM (spring-boot-dependencies)?

`implementation(platform('org.springframework.boot:spring-boot-dependencies:3.2.0'))`. Or `enforcedPlatform()` for strict version enforcement. Then declare dependencies without versions.

Q20. How does version catalog compare to Maven's parent POM dependencyManagement?

Version catalog provides type-safe accessors and centralised version management. Maven's `dependencyManagement` is inherited via parent POM. Version catalogs can be shared across root projects; Maven parent POM is tied to the project hierarchy.

Q21. Which tool is preferred for new projects in 2024?

Gradle for new projects — especially Spring Boot (Spring Initializr defaults to Gradle), Android, and performance-sensitive CI environments. Maven for organisations with existing Maven infrastructure, strict conventions, or teams unfamiliar with Groovy/Kotlin.

Q22. What is the biggest challenge when migrating from Maven to Gradle?

Understanding the configuration vs execution phase distinction, adapting to Gradle's lazy evaluation model, handling complex Maven plugin configurations (especially custom lifecycle bindings), and ensuring build reproducibility with the new tool.

Q23. How does incremental compilation work in Gradle vs Maven?

Gradle: built-in incremental Java compilation — only recompiles classes affected by changes (via dependency analysis). Maven: requires `maven-compiler-plugin` with `useIncrementalCompilation=true` — less sophisticated, often recompiles entire source set.

Q24. What is the Maven equivalent of Gradle's up-to-date checks?

Maven has no direct equivalent. Some plugins check timestamps, but there's no systematic input/output hashing. Maven always re-executes goals unless using the `build-cache` extension or Takari's lifecycle.

Q25. How do configuration profiles work differently?

Maven profiles are XML sections with their own dependencies/plugins/properties, activated by `-P` flag or conditions. Gradle profiles are typically implemented with `if` blocks, `-P` properties, or separate convention plugins — more flexible but less declarative.